ROBO 5000 (CSCI 5202) - Intro to Robotics – Challenge Problem 2 – Group ROBO1

Sawyer McKenny, Pradeep Alapati, Richard Thompson

## To Run:

1. Download code from https://github.com/sawyermckenney/CP2_ROBO1
2. Run CP2_startercode.py
3. Program will run with no user input

## Overview

Challenge problem two involved implementing a RRT algorithm with a 50% goal bias simulating a 2D arena with walls and block obstacles. The RRT algorithm builds a tree structure in the search area (in our case it's a 2D grid) by starting from a root and randomly sampling points. At each step the algorithm finds the nearest node in the tree to the sample and then tries to extend the tree towards the sample if there is no collision.

When we first tasked with the project, we broke it up into two separate parts. We built a tree data structure and implemented the RRT search algorithm. The tree we used in a node class to store each state, position, orientation, and how it goes there. We also had a tree class that manages adding the nodes, finding the nearest node to a given sample and handles the backtracking from the node to the root to ensure a solution path would be found eventually.

## Implementation

`rrt_treey.py`: This is where we implemented the tree class. The tree is made up of nodes (linked list) and each node stores position, orientation, a reference to its parent node, the action it took, the number of steps taken and the trajectory (basally how we track where the robot explored) of points traveled. The start node or the root node is the starting state for the robot, so it has no parent or action. When a new node is added, it is added to a list and given an id (aka index). The approach we took to find the nearest node to a sample point was by iterating through all the nodes and returning the one with the smallest Euclidean distance (per the instructions). Then to extract the solution path, the tree backtracks from the goal node to the root by following the parent references (that are stored in each node). We are able to do this because each node tracks the action and position, and parent.

`rrt_path.py`: The is where the real meat of the problem takes place. The RRT search algorithms start by adding the start/root node as the start of the tree. Then at each iteration either the goal or a random point is picked in the arena with a 50/50 change of either one

happening as per the write up. Then we find the nearest node in the tree and proceed to run up to 50 Monte Carlo rollouts. Each time a left and right wheel and velocity are all chosen at random. The robot is then taken to the starting node and simulates that specific roll out and if that rollout hits an obstacle, it is disregarded. But once five valid rollouts are found and saved, all are added to the tree. We then teleport the bot to the propagation with an end point that is closest to the sample. If we are unable to find five valid rollouts, that sample is skipped. We repeat this process 3000 times or until we land within the goal region.

`CP2_startercode.py`: This is where we simulate the final path of finding the goal. Once the goal is reached via the RRT search, we take the final path via backtracking through the tree. The robot is then brought back to the starting position and since we saved all the sequence of commands in the tree, we can replay it in order to drive the TurtleBot on the solution path that we found.